DEAD CODE DETECTION AND ELIMINATION SYSTEM

COMPILER DESIGN

Mini Project Report

Team Details:

Student Name	KU ID	Enrollment Number	Class
Harshil Dhamat	23BTAI15	202307020033	B.Tech AIML
Vishwa Chokshi	23BTAI14	202307020026	B.Tech AIML
Mitanshu Dholakiya	23BTAI17	202307020035	B.Tech AIML

Institute: Karnavati University

Academic Year: 2026-27

Submission Date: 30/10/26

DECLARATION

We hereby declare that the mini project entitled **"Dead Code Detection & Elimination System"** submitted as part of the course Compiler Design is our original work carried out under the guidance of the faculty member.

We further declare that this work has not been submitted previously to any other institution or university for the award of any degree, diploma, or academic certification.

All research papers, datasets, frameworks, APIs, libraries, websites, and other resources used during the development of the project have been appropriately acknowledged and referenced.

Sr. No.	Student Name	Enrollment No.
1	Vishwa Chokshi	202307020026
2	Harshil Dhamat	202307020033
3	Mitanshu Dholakiya	202307020035

Date: 30/10/2026

Place: Gandhinagar

Abstract

Dead code is a part of a program that does not contribute to the program result or observable behaviour. Keeping unnecessary assignments and computations can make source code harder to understand and may increase the work performed by a program. This project implements an educational Dead Code Detection and Elimination System using C++ in the MSYS2 UCRT64 environment. The system reads a restricted C like input file, represents each statement using a shared data structure, analyses statements in reverse order, identifies certain unused assignments, and writes the retained statements to an optimized output file. The project is divided into parser, detector, optimizer, and main modules so that three team members can develop and test separate parts. The current implementation focuses on simple declarations, assignments, arithmetic expressions, output statements, and return statements. It demonstrates the basic compiler optimization idea of backward liveness analysis while clearly recognizing that full C++ parsing and control flow analysis are outside its current scope.

1. Introduction

A compiler translates a source program into a form that can be executed by a computer. During compilation, optimization techniques improve code by removing unnecessary operations or simplifying computations without changing the intended observable behaviour. Dead code elimination is one such technique. An assignment is a candidate for removal when the value it assigns is never used later and evaluating its expression has no observable side effect.

The project builds a small source statement analyser rather than a complete compiler. It uses C++ standard libraries and can be compiled from the MSYS2 UCRT64 terminal using g++. The output includes a detection report and a separate file containing the retained statements.

2. Problem Statement

Programs can contain assignments whose values are overwritten before being used or are never used at all. Manually locating these statements becomes difficult as code grows. The aim is to develop a simple tool that identifies safely removable assignments in a supported subset of C like statements and generates a cleaner statement file.

3. Objectives

- Read source statements from an input text file.
- Represent statements and variable uses in a common data structure.
- Apply backward liveness analysis to detect selected dead assignments.
- Remove statements marked as dead and preserve the original input file.
- Display the original code, detected statements, reasons, and optimization summary.

4. Scope and Requirements

4.1 Scope

The current version supports one statement per line. It handles basic variable declarations and assignments, simple arithmetic expressions, printf and cout output statements, and return statements. It skips blank lines and lines beginning with a simple single line comment marker. Unknown statements are conservatively retained.

4.2 Software Requirements

- Operating environment: MSYS2 UCRT64 on Windows.
- Programming language: C++17.
- Compiler: g++ from the UCRT64 toolchain.
- Editor: nano or another text editor.
- Input: input.txt containing the supported source statements.
- Output: optimized.txt containing statements not marked as dead.

5. System Design

The system follows a simple pipeline. The parser reads the input file and creates a list of Statement records. The detector scans that list from the last statement to the first and maintains a set of variables whose values may be needed later. The optimizer writes every statement that has not been marked as dead. The main module coordinates these stages and displays the report.

```
input.txt
   |
Parser and Statement Representation
   |
Backward Liveness Based Detector
   |
Optimizer and Report
   |
optimized.txt
```

5.1 Module Description

- parser.h: defines the Statement structure and parser function declaration.
- parser.cpp: reads lines and extracts assignment targets and variable uses.
- detector.h and detector.cpp: declare and implement the dead assignment detection algorithm.
- optimizer.h and optimizer.cpp: write retained statements to the output file.
- main.cpp: runs the stages and displays the results.

5.2 Statement Representation

Each statement record stores the original line number and source text, the assignment target, the right hand side expression, the variables used, and flags describing whether the statement is an assignment, observable, safe to eliminate, or marked as dead. A reason field records why the detector marks an assignment for removal.

6. Algorithm and Working

6.1 Backward Liveness Analysis

A variable is live at a point when its current value may be used later. The detector begins at the end of the statement list and moves backwards. It maintains a set called liveVariables.

- For an observable or unknown statement, add the variables it uses to the live set.
- For an assignment, check whether its target is live after the statement.
- If the target is not live and the expression is considered safe to remove, mark the assignment as dead.
- Otherwise, remove the assignment target from the live set and add variables used by the right hand side.
- After analysis, the optimizer writes all statements that are not marked as dead.

6.2 Pseudocode

```
liveVariables = empty set
for each statement from last to first:
    if statement is an assignment:
        if target is not live and assignment is safe:
            mark statement dead
        else:
            remove target from liveVariables
            add right hand side variables to liveVariables
    otherwise:
        add variables used by statement to liveVariables
write all statements not marked dead
```

6.3 Example

Input statements:

```
int a = 10;
int b = 20;
int c = a + b;
int d = 50;
c = 100;
printf("%d", c);
return 0;
```

The value of c assigned by the third statement is overwritten before it is read. The values assigned to a and b are used only by that removable assignment, so those assignments can also be removed. The value assigned to d is never used. The later assignment to c is retained because printf uses c. The output statement and return statement are retained.

6.4 Expected Optimized Statements

```
c = 100;
printf("%d", c);
return 0;
```

7. Implementation and Execution

7.1 Project Files

The project folder contains parser.h, parser.cpp, detector.h, detector.cpp, optimizer.h, optimizer.cpp, main.cpp, and input.txt. Header files provide shared declarations. The implementation files contain the parsing, detection, and output logic. The main file connects these modules.

7.2 Create the Folder

```
mkdir -p ~/DeadCodeProject
cd ~/DeadCodeProject
```

7.3 Create Files with nano

Create each file using nano followed by the file name. Paste the corresponding source code, save with Ctrl and O, press Enter, and exit with Ctrl and X. Repeat this process for each header and source file.

```
nano parser.h
nano parser.cpp
nano detector.h
nano detector.cpp
nano optimizer.h
nano optimizer.cpp
nano main.cpp
nano input.txt
```

7.4 Compile and Run

Compile all implementation files together from the UCRT64 terminal:

```
g++ -std=c++17 -Wall -Wextra main.cpp parser.cpp detector.cpp optimizer.cpp -o deadcode.exe
```

Run the generated executable:

```
./deadcode.exe
```

The executable reads input.txt in the current folder, displays the original statements and detection report, and creates optimized.txt. The output file is a statement fragment for this educational language subset and is not guaranteed to be a complete standalone C++ program.

8. Team Contribution

Team Member	Assigned Module	Main Responsibility
Vishwa Chokshi	Parser	Read source statements and extract assignment details.
Harshil Dhamat	Detector	Implement backward liveness analysis and mark dead assignments.
Mitanshu Dholakiya	Optimizer	Write retained statements and support output reporting.

9. Testing and Results

The test program contains seven statements. Four assignments are expected to be marked as dead by the example analysis. The assignment c = 100, the printf statement, and the return statement are retained. The report shows the original statement, line number, and reason for removal. The optimizer saves the retained statements in optimized.txt.

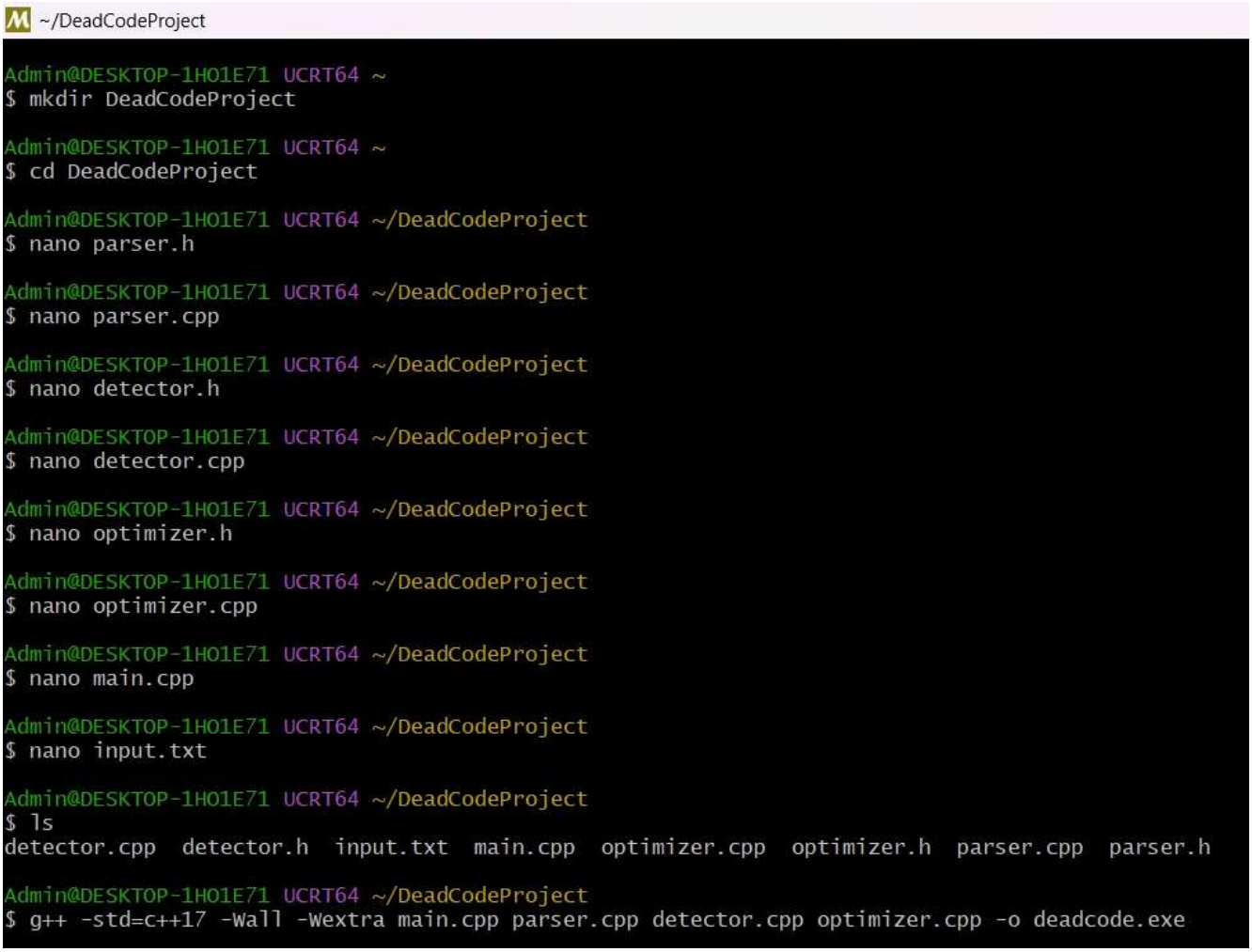
Test	Expected Result	Purpose
Unused assignment	Marked dead when safe	Check unused values
Overwritten assignment	Earlier value may be removed	Check overwritten values
Used assignment	Retained	Check liveness
Output statement	Retained	Preserve observable output
Return statement	Retained	Preserve function return
Unknown statement	Retained conservatively	Avoid unsupported removal

9.1 Result Summary

- Total statements analysed in the sample: 7.
- Dead assignments detected in the sample: 4.
- Statements retained in the sample: 3.
- Output file generated: optimized.txt.

10. Screenshots

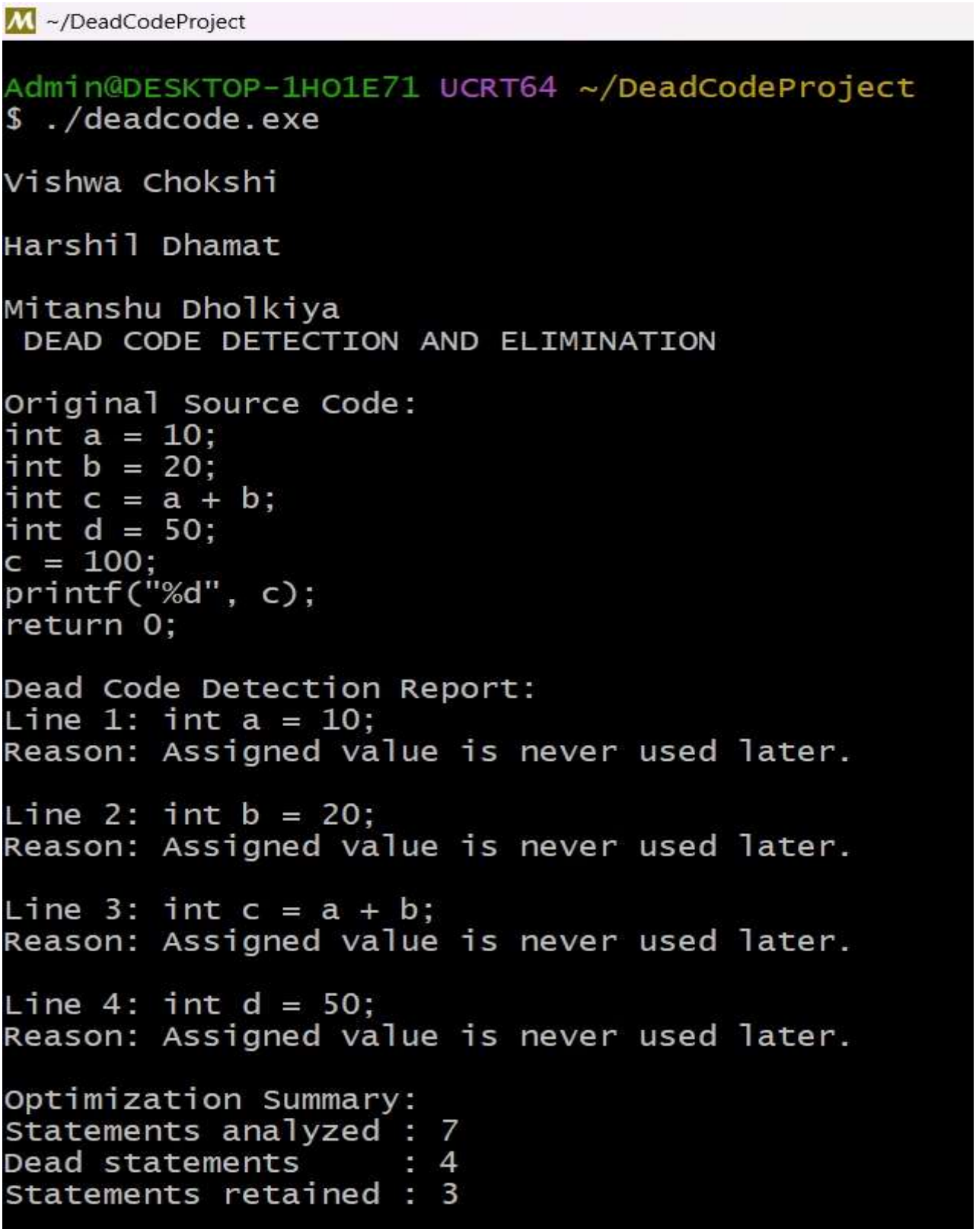
Screenshot_01: Project folder and UCRT64 terminal with g++ compilation command.



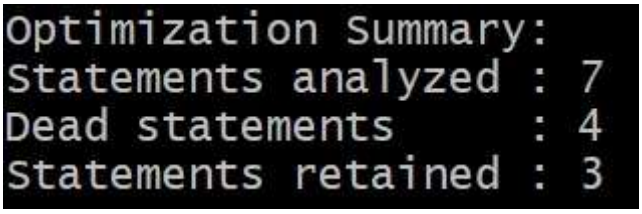
Screenshot_02: The input.txt file in nano.



Screenshot_03: Program output with the original code, dead assignments, and reasons.



Screenshot_04: The optimization Summary



11. Limitations

- The parser supports a restricted C like statement format and is not a complete C++ parser.
- Each statement is expected to occupy one line.
- The current detector does not build a control flow graph and does not analyse branches or loops.
- The current implementation does not perform full type checking or symbol scope resolution.
- Expressions with parentheses are preserved conservatively, but a production compiler must check all possible side effects and language rules.
- The optimized file contains statements only and may need to be placed inside a valid function before compilation.

12. Future Enhancements

- Add lexical analysis and a grammar based parser.
- Support if statements, loops, and control flow graphs.
- Implement reaching definitions and more complete liveness analysis.
- Detect unreachable statements after unconditional return or other control transfer.
- Add constant folding and constant propagation as separate optimizations.
- Compare original and optimized program behaviour using automated tests.

13. Conclusion

The Dead Code Detection and Elimination System demonstrates a fundamental compiler optimization technique using C++ and MSYS2 UCRT64. Its modular structure separates source parsing, backward liveness based detection, and optimized output generation. The sample demonstrates how unused and overwritten assignments can be removed while retaining statements needed for output and program termination. Although the implementation is limited to a small C like language subset, it provides a clear foundation for understanding data flow analysis and can be extended with control flow and additional compiler optimizations.

References

- Aho, Lam, Sethi, and Ullman, Compilers: Principles, Techniques, and Tools, Second Edition, Pearson.
- Keith D. Cooper and Linda Torczon, Engineering a Compiler, Second Edition, Morgan Kaufmann.
- ISO C++ reference, language and standard library documentation, https://en.cppreference.com/
- GCC documentation, https://gcc.gnu.org/onlinedocs/